M19 - Interview, Technical English & Job Launch

LEARNER BOOK - BEGINNER-FIRST TEACHING RC2

Primary teacher

The lesson explains from zero and gives a concrete case before asking for proof. Videos/docs are reinforcement, not a rescue requirement.

Contents

DE19-01 - Truthful one-page Data Engineer resume

What you will be able to do

Build a one-page resume whose every skill/project claim you can prove from completed course evidence.

Why this matters

Interviews expose inflated claims quickly. A smaller truthful resume gives you defendable stories and stronger trust.

Picture it this way

Your resume is a map of evidence, not a wishlist of tools. Every bullet should point to something you actually built, measured or fixed.

Start from zero

- Use clear headline/summary only if it adds relevant context; do not claim seniority you do not have.
- Skills section includes tools you can demonstrate, not videos watched.
- Project bullets use action + system/problem + technical method + validated result/evidence.
- Quantify with real project control totals/performance only; never invent business impact.
- Keep education concise and factual; degree does not need apology.
- One page for early-career is usually easier to scan; tailor keywords truthfully per role.

Teacher demonstration / case

```
Weak: "Expert in Spark, Kafka, Azure and Airflow."
Strong: "Built a PySpark batch pipeline with explicit schemas, guarded joins and Parquet output; valida
ted 18 source rows, 16 completed rows and 25,230 gross-sales total in the course project."
```

Read it like English

- The strong bullet is specific, verifiable and bounded to the project.
- It does not claim production scale that was not experienced.

Expected check

resume claim	proof source
SQL/PostgreSQL	queries/model project
Python ingestion	M05/M08 project
Spark	M12 project controls
Airflow/Docker	M10 live/static evidence
Cloud/Fabric	label offline/live accurately

Common beginner traps

- Listing every course technology as expert.
- Inventing 40% performance improvement.
- Hiding degree instead of presenting relevant transition evidence.

Now you do a similar task

Use job_launch_templates to create one-page resume. For every bullet add a private evidence link/path that proves the claim.

How to prove it

- Save the actual artifact/answer/config/code.
- Save independent evidence/checks.
- State assumption/tradeoff/limitation.
- If you used simulator/offline work, label it honestly.

STOP - check your understanding

If you used an offline Fabric simulator but no live Fabric capacity, what wording keeps the resume truthful?

Answer in your own words before continuing.

DE19-02 - GitHub and project portfolio final polish

What you will be able to do

Make GitHub projects reproducible, readable and evidence-rich for a recruiter/interviewer without publishing secrets or reference solutions.

Why this matters

A repository is stronger than screenshots when another person can understand/run it and see decisions/tests.

Picture it this way

Your portfolio repo is a small product handoff: README is the front door, source/tests are the machinery, evidence shows it works.

Start from zero

- Pin/select 2-4 strongest projects, not every exercise.
- README: problem, architecture, setup, run, data contract, validation, design decisions, limitations.
- Keep synthetic/safe sample data; no secrets/real PII.
- Clean git history is nice but reproducibility/test evidence matters more than perfect aesthetics.
- Do not publish course reference solutions/review answers as if independently authored. Publish your independent project implementation.
- Add architecture diagram only when it clarifies flow.

Teacher demonstration / case

```
README sections:
1 Problem / requirements
2 Architecture/data flow
3 Tech + why
4 Setup/run
5 Validation/control totals
6 Failure/replay behavior
7 Tradeoffs/limitations
8 Evidence/screenshots only as supplement
```

Read it like English

- A reviewer can understand project in a few minutes.
- Validation section differentiates engineering from a code dump.

Expected check

repo check	pass
README run steps	tested clean clone
secrets	none
tests/checks	pass evidence
limitations	truthful
reference solution	not published as own work

Common beginner traps

- Uploading giant course ZIP with no narrative.
- Committing .env/tokens.
- README says production-ready when runtime was only simulated.

Now you do a similar task

Audit one strongest repo using clean clone. Rewrite README so a stranger can reproduce and defend the result.

How to prove it

- Save the actual artifact/answer/config/code.
- Save independent evidence/checks.
- State assumption/tradeoff/limitation.
- If you used simulator/offline work, label it honestly.

STOP - check your understanding

What makes a GitHub repo stronger interview evidence than a screenshot of successful output?

Answer in your own words before continuing.

DE19-03 - LinkedIn/professional profile without overclaiming

What you will be able to do

Create a concise professional profile that matches your resume/evidence and invites relevant data-engineering/analytics conversations without inflated titles.

Why this matters

Recruiters compare LinkedIn, resume and GitHub. Contradictions reduce trust.

Picture it this way

Profile is the public summary of the same evidence map, not a different fictional career.

Start from zero

- Headline can state target + concrete skills/project focus without "Senior"/years not earned.
- About section: current transition, strongest technical themes, proof/projects, target roles.
- Experience/project entries clearly label personal/course projects vs paid employment.
- Skills prioritize role-relevant proven abilities.
- Use same dates/names/education facts as resume.
- Do not display private contact/address/credentials unnecessarily.

Teacher demonstration / case

```
Headline example pattern:
Aspiring / Junior Data Engineer | Python, SQL, PostgreSQL, Spark | Building reliable batch & ingestion
projects
# Adapt only to skills you can defend.
```

Read it like English

- The wording signals target without pretending current employment.
- Profile should link to selected portfolio project(s).

Expected check

profile element	truth test
title/headline	target/proven skill, not invented job
project	clearly personal/course
skills	demo-able
links	public safe repo

Common beginner traps

- Calling yourself Data Engineer at Company X when not employed there.
- Copying keyword soup.
- LinkedIn claims Azure/Fabric live expertise when only offline concepts completed.

Now you do a similar task

Fill the supplied profile template and cross-check every claim against resume/project evidence.

How to prove it

- Save the actual artifact/answer/config/code.
- Save independent evidence/checks.
- State assumption/tradeoff/limitation.
- If you used simulator/offline work, label it honestly.

STOP - check your understanding

Why can "targeting Data Engineer roles" be more credible than simply setting your title to "Data Engineer" before employment?

Answer in your own words before continuing.

DE19-04 - Degree-aware job targeting: apply/skip rules

What you will be able to do

Use an apply/skip/conditional rubric for job descriptions: required core skills, experience level, degree filters, location/interview constraints and learnable gaps.

Why this matters

Beginners waste time either applying to everything or rejecting themselves for every non-match. A rubric creates consistent decisions.

Picture it this way

Treat a job description as requirements triage, not a perfect checklist or a command to self-reject.

Start from zero

- Separate must-have signals from wish-list/preferred language.
- Apply when core work matches and gaps are realistically learnable; do not require 100%.
- Skip/low priority when role is clearly senior/lead, hard legal clearance, incompatible location/work authorization or mandatory degree/license you cannot satisfy.
- Degree requirements can be strict or flexible depending employer; read wording and evidence from application outcome.
- Track recurring gaps to update study plan rather than adding every keyword instantly.
- Tailor resume truthfully to relevant terms, never fake experience.

Teacher demonstration / case

```
JD rubric (0/1/2):
core SQL/Python/data pipelines
junior/experience range
cloud/tool overlap
location/interview feasible
degree wording
portfolio fit
hard blockers
=> Apply / Conditional / Skip
```

Read it like English

- The scoring is a decision aid, not certainty about recruiter filters.
- A conditional role may be worth applying when core fit is high and one tool differs.

Expected check

JD wording	response
"3+ preferred"	may still apply if junior/core fit
"Lead/8+ years"	usually skip as junior
"B.Tech mandatory"	possible hard filter; prioritize alternatives
"degree or equivalent experience"	apply if skills/projects fit

Common beginner traps

- Rejecting self for every missing tool.
- Applying to senior architect roles for volume.
- Changing resume education facts.

Now you do a similar task

Use the 15-JD targeting drill/template. Classify each Apply/Conditional/Skip and write one-sentence reason.

How to prove it

- Save the actual artifact/answer/config/code.
- Save independent evidence/checks.
- State assumption/tradeoff/limitation.
- If you used simulator/offline work, label it honestly.

STOP - check your understanding

What is the difference between a tool gap and a hard blocker?

Answer in your own words before continuing.

DE19-05 - SQL interview drills

What you will be able to do

Solve SQL interview questions by stating grain/key, writing a correct simple query, checking edge cases and explaining it aloud.

Why this matters

Interviewers often evaluate reasoning and correctness more than clever syntax.

Picture it this way

Treat the prompt like a mini data model: define what one output row means before typing joins/aggregates.

Start from zero

- Clarify tables/grain/keys/null/duplicate assumptions.
- Start with readable CTEs/subqueries; optimize only after correctness.
- Joins: predict cardinality and validate duplication.
- Aggregations: output grain + WHERE vs HAVING.
- Windows: partition/order/frame meaning.
- Top-N/ties/latest-row require deterministic tie rule.
- Explain complexity/performance/index ideas only after correct query.

Teacher demonstration / case

```
-- Prompt: latest order per customer
WITH ranked AS (
  SELECT o.*,
         ROW_NUMBER() OVER(
           PARTITION BY customer_id
           ORDER BY order_date DESC, order_id DESC
         ) AS rn
  FROM orders o
)
SELECT * FROM ranked WHERE rn=1;
```

Read it like English

- Partition = one ranking sequence per customer.
- Tie-breaker order_id makes winner deterministic when dates tie.
- Output grain = one row/customer (assuming customer with orders).

Expected check

interview step	say/do
clarify	grain/key/ties/nulls
solve	readable SQL
validate	sample/tie/duplicate
explain	why each clause
improve	index/scale only if asked

Common beginner traps

- Writing memorized query before clarifying ties.
- Using DISTINCT to hide join duplication.
- Silent for five minutes then only final SQL.

Now you do a similar task

Do timed SQL drills from template: 10-15 minutes each, then a 60-second explanation and one edge case.

How to prove it

- Save the actual artifact/answer/config/code.
- Save independent evidence/checks.
- State assumption/tradeoff/limitation.
- If you used simulator/offline work, label it honestly.

STOP - check your understanding

Why is a deterministic tie-breaker important in "latest row" interview questions?

Answer in your own words before continuing.

DE19-06 - Python interview drills

What you will be able to do

Solve practical Python data-engineering interview tasks with clear data structures, edge cases, complexity and tests.

Why this matters

Junior DE Python interviews often test transformation/string/list/dict/file reasoning rather than advanced algorithms only.

Picture it this way

Show your working method: define input/output, choose a simple data structure, test normal/boundary cases, then discuss complexity.

Start from zero

- Clarify input size/format/nulls/duplicates/order requirements.
- Use dict/set/Counter for key/count/dedup problems when appropriate.
- Write small functions with explicit return values.
- Test empty/single/duplicate/invalid boundary cases.
- For files/JSON/CSV use parsers/context managers and avoid loading unbounded data unnecessarily.
- Explain O(n), memory and streaming alternative in plain language.

Teacher demonstration / case

```
def count_by_customer(order_rows):
    counts={}
    for r in order_rows:
        cid=r["customer_id"]
        counts[cid]=counts.get(cid,0)+1
    return counts
```

Read it like English

- One pass over n rows -> O(n) time.
- Dictionary stores up to unique customers -> O(k) memory.
- For huge file, rows can be streamed rather than list-loaded.

Expected check

case	test
empty	{}
one row	one key count1
duplicates	count increments
missing customer_id	define reject/error policy

Common beginner traps

- Using pandas for every small interview list problem.
- No edge tests.
- Claiming complexity without explaining variables.

Now you do a similar task

Use python_mock_TODO.py and timed drills. After each, explain data structure, complexity and how you would adapt for a file too large for memory.

How to prove it

- Save the actual artifact/answer/config/code.
- Save independent evidence/checks.
- State assumption/tradeoff/limitation.
- If you used simulator/offline work, label it honestly.

STOP - check your understanding

When is a dictionary the natural choice for an interview data problem?

Answer in your own words before continuing.

DE19-07 - Data-engineering concepts and troubleshooting drills

What you will be able to do

Answer data-engineering concept/troubleshooting questions using a repeatable structure: define, diagnose from evidence, repair smallest layer, prove.

Why this matters

Interviews often test whether you understand real failure modes rather than can recite tool definitions.

Picture it this way

Answer like an on-call engineer: symptom -> evidence -> hypotheses -> targeted check -> fix -> validation/prevention.

Start from zero

- For duplicates: clarify grain/key/replay/join before dropDuplicates.
- For slow Spark: inspect plan/shuffle/skew/input size before adding workers.
- For Airflow failure: exact DAG run/task log -> classify -> idempotent retry.
- For Kafka lag: per-partition lag, consumer/sink/skew/rebalance before reset offsets.
- For missing data: source/window/state/row reconciliation.
- For schema drift: observed vs contract, compatibility/quarantine.
- Conclude with proof/monitoring to prevent recurrence.

Teacher demonstration / case

```
Troubleshooting answer template:
1. Clarify symptom/impact/window.
2. Check run/task/source/target evidence.
3. Narrow likely layer.
4. Repair smallest cause safely.
5. Rerun/replay only if idempotent.
6. Validate controls.
7. Add test/alert if appropriate.
```

Read it like English

- This structure applies across tools and prevents random guess lists.

Expected check

prompt	first evidence
Airflow task failed	task log/run ID
Spark slow join	plan + skew/task metrics
Kafka lag	partition lag + consumer/sink
target missing rows	source-target reconciliation/state

Common beginner traps

- Answering with ten possible causes and no order.
- "Restart service" as first step.
- No validation after fix.

Now you do a similar task

Use G06 troubleshooting/project-defense drill. For five incidents, give a 90-second structured answer with first three checks in order.

How to prove it

- Save the actual artifact/answer/config/code.
- Save independent evidence/checks.
- State assumption/tradeoff/limitation.
- If you used simulator/offline work, label it honestly.

STOP - check your understanding

What makes a troubleshooting answer stronger than listing every possible cause?

Answer in your own words before continuing.

DE19-08 - Project walkthrough interview

What you will be able to do

Walk through a portfolio project in 3-5 minutes from problem -> architecture -> implementation -> validation -> failure/replay -> tradeoff without drowning in code.

Why this matters

A strong project walkthrough proves you understand why you built it, not just that files exist.

Picture it this way

Tell the story of the system like a guided factory tour: business input, flow, safeguards, evidence and what you would improve.

Start from zero

- Start with problem/SLA/source/consumer.
- Draw/describe data flow and important grain/key/state.
- Highlight 2-3 technical decisions and alternatives.
- Show validation/control totals and one failure/replay test.
- State limitations honestly: local scale, simulated cloud, no live production on-call, etc.
- End with what you would change at larger scale/production.

Teacher demonstration / case

```
3-minute outline:
0:00 problem + requirements
0:30 architecture/data flow
1:15 key implementation decisions
2:00 correctness + failure/replay evidence
2:40 tradeoff/limitation + next scale step
```

Read it like English

- This prevents spending all time on library names.
- Interviewers can then drill into any decision.

Expected check

walkthrough	evidence
problem	why project exists
architecture	data flow/grain/state
decision	why not alternative
validation	counts/tests
failure	retry/replay/incident
limitation	truthful

Common beginner traps

- Starting with "I used Python, SQL, Spark...".
- No business/data grain explanation.
- Pretending local synthetic project handled billions.

Now you do a similar task

Record/write a 3-minute walkthrough for each top portfolio project and answer three likely follow-ups.

How to prove it

- Save the actual artifact/answer/config/code.
- Save independent evidence/checks.
- State assumption/tradeoff/limitation.
- If you used simulator/offline work, label it honestly.

STOP - check your understanding

Which single project detail most clearly proves you personally understand the engineering rather than copied code?

Answer in your own words before continuing.

DE19-09 - Architecture/system-design defense for junior roles

What you will be able to do

Handle junior system-design prompts by clarifying requirements and drawing a simple defendable data flow with reliability/security/cost tradeoffs.

Why this matters

Interviewers do not expect senior-scale mastery from junior candidates; they want structured reasoning and appropriate scope.

Picture it this way

Design a small bridge correctly before proposing a six-level megastructure.

Start from zero

- Clarify source, volume, latency, retention, consumers and sensitive data.
- Choose batch/stream from SLA.
- Define raw landing/replay, transform compute, serving store and orchestration.
- Define state/idempotency/reconciliation/failure path.
- Add IAM/secrets/PII and monitoring.
- Mention scale bottleneck/cost driver and one future evolution.
- Avoid unnecessary Kafka/Kubernetes if requirements do not demand them.

Teacher demonstration / case

```
Prompt: "Design daily order analytics"
Source API/files -> immutable raw object storage
-> batch transform (Python/Spark based on scale)
-> warehouse/lakehouse fact/dims
-> BI
Orchestrator schedules window; state after publish; row/amount controls; IAM/alerts.
```

Read it like English

- The design is intentionally boring/appropriate for daily SLA.
- If interviewer changes SLA to seconds, revisit source/streaming architecture.

Expected check

requirement change	design impact
daily -> 5 sec	consider CDC/Kafka/stream
10 GB -> 5 TB/day	distributed compute/layout
PII introduced	access/minimization/security
RTO 4h -> 5m	redundancy/recovery

Common beginner traps

- Name-dropping every tool.
- No raw/replay path.
- No failure/idempotency.

Now you do a similar task

Use G07 system-design drill. Solve two prompts in 15 minutes each, then defend one rejected technology.

How to prove it

- Save the actual artifact/answer/config/code.
- Save independent evidence/checks.
- State assumption/tradeoff/limitation.
- If you used simulator/offline work, label it honestly.

STOP - check your understanding

Why can choosing a simpler batch design be a strong answer rather than a weak one?

Answer in your own words before continuing.

DE19-10 - Behavioral answers and career-transition story

What you will be able to do

Build behavioral answers from real Situation-Task-Action-Result-Learning evidence, including career transition and mistakes, without inventing employment stories.

Why this matters

Behavioral interviews evaluate communication, ownership, learning and collaboration. Fabricated stories are risky and unnecessary.

Picture it this way

Use real study/project situations as small but truthful evidence of persistence, debugging and improvement.

Start from zero

- STAR: Situation context, Task responsibility, Action you took, Result evidence.
- Add Learning/what you would do differently.
- Use course/project/Git collaboration/learning incidents honestly and label context.
- Career-transition story: past education/work context -> why data -> evidence of structured upskilling/projects -> target role.
- Failure story should show ownership/diagnosis/change, not blame.
- Keep answer 1-2 minutes and answer the actual question.

Teacher demonstration / case

```
Prompt: "Tell me about a difficult technical problem."
S: project output duplicated on rerun.
T: make pipeline replay-safe.
A: checked business key/state, changed load to upsert, added rerun test/reconciliation.
R: second run kept stable key/count.
L: design idempotency before enabling retries.
```

Read it like English

- This can be a personal project story if stated honestly.
- Result is technical evidence, not invented revenue impact.

Expected check

behavior prompt	evidence source
learning quickly	course/project transition
mistake	debug/repair episode
ownership	independent practical
conflict/team	real collaboration only; do not invent

Common beginner traps

- Inventing team conflict if you worked alone.
- Five-minute autobiography.
- Result with fake business percentage.

Now you do a similar task

Write six STAR-L stories from real experiences/projects and mark which interview question each can answer.

How to prove it

- Save the actual artifact/answer/config/code.
- Save independent evidence/checks.
- State assumption/tradeoff/limitation.
- If you used simulator/offline work, label it honestly.

STOP - check your understanding

How can a course project provide a valid behavioral example without pretending it was paid production work?

Answer in your own words before continuing.

DE19-11 - Technical English: explain code, failures and decisions clearly

What you will be able to do

Explain technical ideas in simple structured English: purpose -> input/output -> mechanism -> evidence -> failure/tradeoff.

Why this matters

Strong technical communication can compensate for imperfect fluency; clarity matters more than accent or complicated vocabulary.

Picture it this way

Explain as if handing the system to a teammate who needs to operate it tomorrow.

Start from zero

- Use short sentences and concrete nouns/verbs.
- Define one term before using many acronyms.
- State row grain/key/window when explaining data transformations.
- When stuck, say what you know, assumption and how you would verify rather than filling silence with guesses.
- Use signposts: "First... Then... The reason is... I validate it by..."
- Practise speaking SQL/Python/project decisions aloud, not only reading.

Teacher demonstration / case

```
60-second explanation pattern:
"This pipeline reads ___. One input row means ___.
First I ___. Then I ___. I use ___ because ___.
The target grain is ___. I validate it with ___ and ___.
If ___ fails, I ___ rather than ___."
```

Read it like English

- The structure makes technical English predictable.
- Correct simple wording is better than memorized jargon.

Expected check

communication	good habit
unknown	state assumption + verification
decision	because + tradeoff
proof	specific check
failure	ordered troubleshooting

Common beginner traps

- Trying to sound advanced with jargon.
- Never saying row grain/key.
- Pretending to know unknown answer.

Now you do a similar task

Choose five course topics and record/write 60-second explanations using the template. Remove jargon that you cannot define simply.

How to prove it

- Save the actual artifact/answer/config/code.
- Save independent evidence/checks.
- State assumption/tradeoff/limitation.
- If you used simulator/offline work, label it honestly.

STOP - check your understanding

What can you say in an interview when you genuinely do not know a tool-specific detail?

Answer in your own words before continuing.

DE19-12 - Application tracker, feedback loop and weekly repair cycle

What you will be able to do

Run job search as a measurable feedback loop: applications -> responses -> interviews -> failure categories -> targeted repair.

Why this matters

Random high-volume applying hides whether resume, targeting or interview skill is the real bottleneck.

Picture it this way

Treat job search like a pipeline with conversion metrics and defect categories, not a daily mood score.

Start from zero

- Track date, company, role, JD link/snapshot, fit score, application channel, status and next action.
- Track funnel rates: applications -> recruiter screens -> technical -> offers.
- Low screen rate suggests targeting/resume/profile gap; technical failures suggest SQL/Python/DE topic repair.
- Record exact interview questions/gaps immediately after interview without confidential company material.
- Weekly review chooses 1-2 repair themes, not 20 new technologies.
- Follow up professionally when appropriate; do not spam recruiters.

Teacher demonstration / case

```
Weekly review:
applications 30
screens 4
technical 2
offers 0
notes: SQL windows weak in both technicals
next week: keep targeted applications + 3 window-function mocks + project walkthrough practice
```

Read it like English

- The example converts outcome into a specific repair loop.
- A small sample is noisy; do not overreact to one rejection.

Expected check

funnel symptom	possible focus
0 screens after many good-fit apps	resume/targeting/profile
screens, no technical pass	technical/story
technical pass, no final	behavioral/fit/competition
many hard blockers	target role/company mix

Common beginner traps

- Applying 200 irrelevant jobs to hit quota.
- Changing career path after one rejection.
- No record of interview questions.

Now you do a similar task

Create application tracker template and weekly review. Define thresholds that trigger resume audit vs SQL repair vs targeting change.

How to prove it

- Save the actual artifact/answer/config/code.
- Save independent evidence/checks.
- State assumption/tradeoff/limitation.
- If you used simulator/offline work, label it honestly.

STOP - check your understanding

Why is application count alone a poor success metric?

Answer in your own words before continuing.

DE19-13 - Timed coding test: arrays, strings and hash-map problems

What you will be able to do

Solve common timed coding patterns (arrays/strings/hash maps) with calm problem decomposition and correctness tests.

Why this matters

Some DE interviews include general coding screens. You do not need competitive-programming tricks for every role, but basic data-structure fluency helps.

Picture it this way

Recognize a few reusable tools: scan once, hash set/map, two pointers, sorting tradeoff.

Start from zero

- Clarify input/return and constraints.
- Start with brute force if needed, then improve.
- Hash set/map solves membership/frequency/lookup in average O(1).
- Two pointers often helps sorted arrays/strings.
- Test empty, one element, duplicates, negative/case/whitespace as relevant.
- Communicate complexity and tradeoff.
- If stuck, write a correct simple solution before chasing optimal.

Teacher demonstration / case

```
        def first_duplicate(values):
            seen=set()
            for x in values:
                if x in seen:
                    return x
                seen.add(x)
            return None
```

Read it like English

- One scan -> O(n) average time, O(n) memory.
- Returns first duplicate by encounter order.
- Empty/no-duplicate returns None.

Expected check

pattern	tool
frequency/group count	dict/Counter
seen duplicate	set
lookup complement	dict/set
sorted pair scan	two pointers
top k	sort/heap depending constraints

Common beginner traps

- Memorizing LeetCode code without explaining.
- No tests under timer.
- Optimizing before a working baseline.

Now you do a similar task

Do 5 timed problems (15-20 min each) from arrays/strings/hash-map categories and write post-mortem: pattern, bug, final complexity.

How to prove it

- Save the actual artifact/answer/config/code.
- Save independent evidence/checks.
- State assumption/tradeoff/limitation.
- If you used simulator/offline work, label it honestly.

STOP - check your understanding

Why is a correct O(n log n) solution sometimes better in an interview than an unfinished O(n) idea?

Answer in your own words before continuing.

DE19-14 - Combined SQL + Python interview simulation

What you will be able to do

Complete a combined SQL + Python interview simulation with time management, explanation and self-scoring/repair plan.

Why this matters

Real interviews switch contexts. The final simulation tests technical correctness plus communication under time.

Picture it this way

Treat it like a flight simulator: timed, closed notes, realistic handoff, then inspect mistakes after landing.

Start from zero

- Use a fixed block: SQL 25-30 min, Python 25-30, DE concepts/project 15-20, explanation/review.
- Clarify questions before coding.
- Write/run mental/sample tests and explain assumptions.
- If stuck, communicate partial approach and move rather than burning all time.
- After mock, score correctness, speed, explanation and edge-case handling separately.
- Repair only the weakest 1-2 areas and run a fresh mock, not the same questions immediately.

Teacher demonstration / case

```
Mock scorecard (0-5 each):
SQL correctness __
SQL explanation __
Python correctness __
Python complexity/tests __
DE troubleshooting __
Project/system design __
Technical English clarity __
Time management __
```

Read it like English

- Separate scores tell you whether the problem is knowledge or communication/time.
- Fresh retest prevents memorized answer from appearing as improvement.

Expected check

score pattern	repair
correct but slow	timed repetition
wrong joins/windows	SQL concept drills
Python bugs	small pattern tests
good technical, unclear speech	60-sec explanations
project vague	walkthrough/validation evidence

Common beginner traps

- Practising only comfortable questions.
- Repeating same mock until memorized.
- Self-scoring only pass/fail.

Now you do a similar task

Run the combined mock closed-note. Save answers, scorecard and 3 concrete next actions, then schedule/use a fresh question set after repair.

How to prove it

- Save the actual artifact/answer/config/code.
- Save independent evidence/checks.
- State assumption/tradeoff/limitation.
- If you used simulator/offline work, label it honestly.

STOP - check your understanding

Why should the retest use different questions instead of repeating the same mock?

Answer in your own words before continuing.